Gudotopia Defense Guide

Nov 12, 2026 · @Koro

Start here

Gudotopia is a Java 17 Swing gardening game: 34 source files, about 3,400 lines, no external libraries. You water soil, plant random seeds, wait for them to grow, and harvest the flowers for money before they wither.

The code has four layers, and each layer only talks to the one below it:

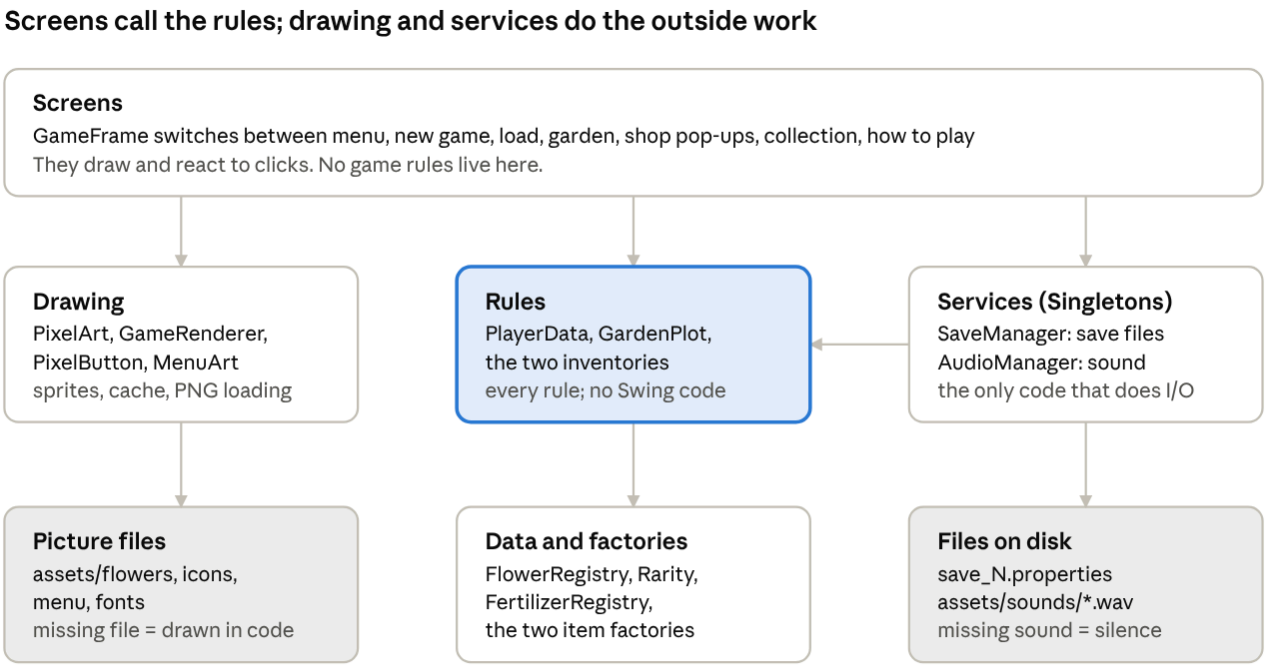
1. **Screens** draw things and react to clicks. They contain no game rules.
2. **Rules** live in one class, `PlayerData`. It has no Swing code, which is why it can be tested without a window.
3. **Data tables** list what exists in the game: the 12 flowers, the 3 rarity tiers, the 2 fertilizers.
4. **Services** do the outside-world work: `SaveManager` reads and writes save files, `AudioManager` plays sound.

If a panelist asks "where is X decided?", the answer is almost always `PlayerData` for a rule, a registry for a number, and a panel for how it looks.

Every section below names the exact file and method, so you can open it during the defense. The last section has likely questions with short answers and a suggested split of topics for seven members.

File map

All 34 files are in `src/`, one class per file, in the default package. Sorted by layer, top to bottom.



Tinted boxes are files, not code. The highlighted box is where every game rule lives.

code layers · 5 groups of classes, 2 groups of files

Arrows mean "calls". Screens never touch a file directly, and the rules never draw; `SaveManager` is the one service that reads `PlayerData` back out to disk.

File	Layer	What it is for
`Main`	Start	Entry point. Opens the save folder, creates the window.
`GameFrame`	Screens	The window. Switches screens with a `CardLayout` and decides when to save, load, delete and exit.
`ScreenPanel`	Screens	Parent of every screen. Paints the sky and clouds and runs the 30-frames-per-second timer.
`MainMenuPanel`	Screens	Title screen with NEW, LOAD, HOW TO PLAY, EXIT.
`HowToPlayPanel`	Screens	Instructions. Prices are read from `Rarity`, so they never go out of date.
`NewGamePanel`	Screens	Type a garden name and create a save.
`LoadGamePanel`	Screens	List of saves with LOAD and a two-click DELETE.
`GardenPanel`	Screens	The gameplay screen: the grass block, plots, plants, tooltip, message banner, bottom bar, and what a click on a plot does.

File	Layer	What it is for
`ShopPanel`	Screens	The six buttons on the right of the garden.
`ItemDialog`	Screens	The shop pop-up shared by seeds and fertilizer (abstract class).
`SeedDialog`	Screens	Seed version of the pop-up.
`FertilizerDialog`	Screens	Fertilizer version of the pop-up; clicking an item selects it as a tool.
`FlowerCollectionPanel`	Screens	Grid of all flowers; undiscovered ones are silhouettes.
`PixelButton`	Drawing	The green custom-painted button used everywhere.
`GameRenderer`	Drawing	Shared drawing helpers: text with outline, boxes, buttons, hills, clouds, word wrap, the plant on a plot.
`PixelArt`	Drawing	Makes every sprite: draws them in code or loads a PNG, and caches the result.
`MenuArt`	Drawing	Loads the main-menu pictures from `assets/menu`.
`PlayerData`	Rules	One save's progress and **all game rules**: buy, water, fertilize, plant, harvest, expand. No Swing code.
`GardenPlot`	Rules	One square of soil: watered or not, its fertilizer, its flower, growth so far, death deadline.
`SeedInventory`	Rules	Seeds you own, and the random pick when planting.
`FertilizerInventory`	Rules	Fertilizers you own.
`GameConfig`	Rules	Timings: 30 s to harvest, 10 s warning, autosave every 15 s.
`Flower`	Data	One flower's definition: id, name, rarity, colours. Immutable.
`Rarity`	Data	The three tiers with drop chance, sell price and growth time, and the roll.
`FlowerRegistry`	Data	The table of the 12 flowers and the random-seed pick.
`Fertilizer`	Data	One fertilizer's definition and its speed bonus.

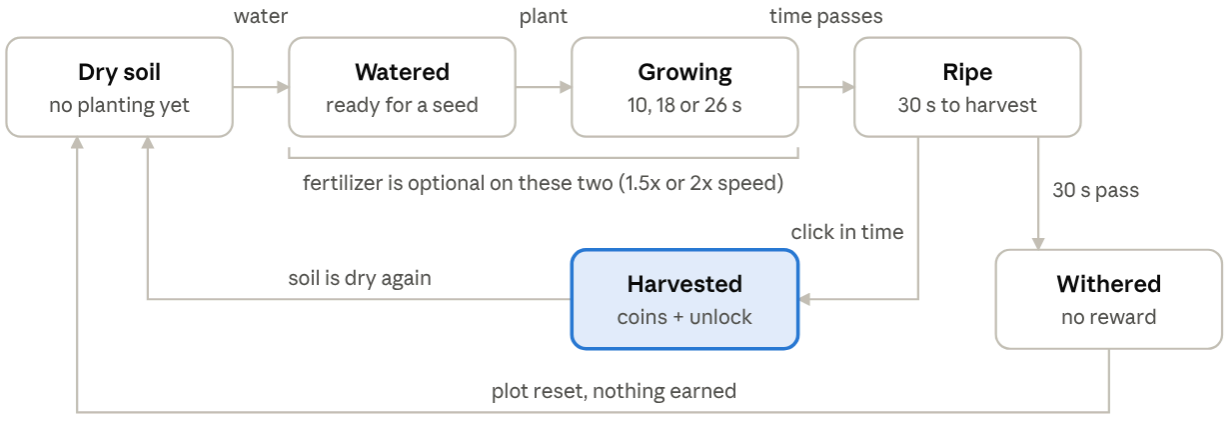
File	Layer	What it is for
`FertilizerRegistry`	Data	The table of the 2 fertilizers and their drop table.
`ItemFactory`	Pattern	Factory Method interface: `create(Random)`.
`SeedFactory`	Pattern	Creates a random seed.
`FertilizerFactory`	Pattern	Creates a random fertilizer.
`SaveManager`	Service	Singleton. The only class that touches save files: list, create, load, save, delete.
`SaveInfo`	Service	One row of the Load list (a record).
`SaveException`	Service	Error whose message can be shown to the player as it is.
`AudioManager`	Service	Singleton. Background music and sound effects.

The three largest files are `GardenPanel` (528 lines), `PixelArt` (400) and `GameRenderer` (276). All three are drawing code; the rules class `PlayerData` is 209 lines.

Game mechanics

A flower needs three steps in a fixed order: water the soil, optionally fertilize, then plant. Every rule below is a method in `PlayerData`, and each returns a `Result` holding `ok` and a `message`. The screen only shows that message; it never decides anything itself.

A plot always returns to dry soil, with or without a reward



life of one plot · 6 states, 2 endings

A ripe flower has two endings. Click it within 30 seconds and it pays out; wait too long and it withers for nothing. Either way the plot must be watered again.

Action	Method	Rule
Buy a seed	`buySeed()`	Costs $2. You get one random flower seed.
Buy fertilizer	`buyFertilizer()`	Costs $10. You get one random fertilizer.
Water	`water(r, c)`	Only an empty, dry plot.
Fertilize	`applyFertilizer(r, c, f)`	A watered empty plot or a growing flower. One per plot. Used up only if it is really applied.
Plant	`plant(r, c)`	Needs a watered empty plot and at least one seed. A random seed from your bag is planted.
Harvest	`harvest(r, c)`	Only a fully grown flower. Adds its price, unlocks it in the collection, and the soil goes back to dry.
Larger plot	`expandGarden()`	3x3 to 4x4 costs $50, then $100, then $200. Maximum 6x6.

You start with $100 and a 3x3 garden.

What a click on a plot does

`GardenPanel.clickCell` picks the action in this order:

1. The flower is fully grown: harvest it, whatever tool is in your hand.
2. The WATER tool is on: water the plot.
3. A fertilizer is selected: apply it.
4. No tool and the plot is empty: plant.
5. Otherwise: show "still growing".

How growth works without a running timer

Growth is calculated from real timestamps, not counted by a timer. Each `GardenPlot` stores three numbers: `growthMs` (growth so far), `updatedMs` (when that was last calculated) and `deathDeadlineMs`.

`GardenPlot.advance(now)` does the work:

1. Time passed = `now - updatedMs`.
2. Growth added = time passed x speed. Speed is 1.0, or 1.5 with Worm, or 2.0 with Manure.

3. If the flower finished growing inside that interval, it works out the exact moment it finished and sets the deadline to that moment plus 30 seconds.
4. If `now` is past the deadline, it returns `DIED`. `PlayerData.update()` then resets the plot and gives no reward.

`PlayerData.update()` runs `advance` on every plot. It is called on every frame, before every action, before saving, and right after loading. That last call is why flowers keep growing, and can wither, while the game is closed.

The four growth pictures come from `GardenPlot.stage()`: seed below 15% progress, sprout below 45%, bud after that, and the full flower at 100%. In the last 10 seconds before the deadline the flower droops and shows a red countdown.

All timings are in `GameConfig`. They can be changed without editing code, for example `-Dgudotopia.growthScale=0.25` makes everything grow four times faster for a demo.

The randomizer

Three things are random: which seed you buy, which fertilizer you buy, and which seed gets planted. All three use one `java.util.Random` that `PlayerData` owns and passes in as a parameter.

1. Buying a seed: roll a tier, then pick a flower

The path is `PlayerData.buySeed()` to `SeedFactory.create(rng)` to `FlowerRegistry.randomSeed(rng)`.

Step one is `Rarity.roll(rng)`. It draws a whole number from 0 to 99 and walks the drop table, adding up the percentages until the number falls inside a tier.

```
int n = rng.nextInt(100), running = 0;
for (Rarity r : values()) {
    running += r.dropPercent;
    if (n < running) return r;
}
```

Tier	Chance	Numbers that give it	Sells for	Grows in	Flowers	Chance of one specific flower
Common	70%	0 to 69	$5	10 s	5	14%
Rare	25%	70 to 94	$25	18 s	4	6.25%
Legendary	5%	95 to 99	$100	26 s	3	1.67%

Step two collects every flower of that tier into a list and picks one with `rng.nextInt(pool.size())`, so flowers inside a tier are equally likely.

Two safety checks run when the game starts. `Rarity.validateDropTable()` stops the game if the percentages do not total 100. `FlowerRegistry` stops the game if a tier has no flower, because rolling that tier would crash.

2. Buying fertilizer: the same idea, its own table

`FertilizerRegistry.roll(rng)` uses the same walk over its own table: 70% Worm (+50% growth speed) and 30% Manure (+100%).

3. Planting: weighted by how many you own

`SeedInventory.takeRandom(rng)` treats every single seed as one ticket. It draws a number from 0 to total minus 1, then subtracts each flower's count until the number goes below zero.

With 3 Daisy, 2 Rose and 1 Sunflower there are 6 tickets. Daisy is planted 50% of the time, Rose 33.3%, Sunflower 16.7%.

Why the Random is passed in

Because the methods receive the `Random`, a test can pass `new Random(42)` and get the same sequence every run. `GameTest` buys 100,000 seeds and checks each tier lands within 0.8 points of its percentage, and plants 60,000 times to check the 50 / 33.3 / 16.7 split.

Prices and growth times are not random. They are fixed per tier in `Rarity`.

Saving and loading

Each garden is one plain text file, and `SaveManager` is the only class that reads or writes them. There is no database.

Where and what

Files live in the home folder under `.gudotopia/saves/`, named `save_1.properties`, `save_2.properties` and so on. On Windows that is `C:\Users\<you>\.gudotopia\saves\`. The folder can be moved with `-Dgudotopia.dir=<folder>`, which is how the tests use a temporary folder.

The format is Java's built-in `Properties`: one `key=value` per line. This is a real save from a test game:

```
name=Gudo Garden
money=193
gardenSize=4
createdAt=1794457064125
lastPlayed=1794457064526
collection=brent,daisy,klent,rovid,sunflower,zeus
seed.daisy=1
seed.tulip=2
fertilizer.fert_worm=1
plot.0.1.watered=1
plot.0.1.flower=rose
plot.0.1.fertilizer=fert_manure
plot.0.1.growthMs=4200
plot.0.1.updatedMs=1794457064000
plot.0.1.deadlineMs=0
```

Key	Meaning
`name`, `money`, `gardenSize`	The garden's name, coins, and size (3 to 6).
`createdAt`, `lastPlayed`	Timestamps in milliseconds. `lastPlayed` sorts the Load list.
`collection`	Ids of the flowers discovered so far.
`seed.<id>`, `fertilizer.<id>`	How many of each you own.
`plot.<row>.<col>.*`	One plot's state. Untouched plots are not written.

The three plot timestamps are exactly the three numbers `GardenPlot` needs to continue growth, which is what makes offline growth work.

The five operations

Operation	Method	What happens
List	`listSaves()`	Reads every `save_N.properties`, newest first. A damaged file is skipped, never deleted.
Create	`createSave(name)`	Checks the name, picks the next free number, writes a new file. It can never overwrite an existing one.
Load	`loadSave(id)`	Rebuilds a `PlayerData` from the file, then calls `update()` so the time that passed while the game was closed is applied.

Operation	Method	What happens
Save	`save(data)`	Calls `update()`, turns the game into `Properties`, writes the file.
Delete	`deleteSave(id)`	Removes that one file.

A garden name must be 1 to 20 characters, use only letters, numbers, spaces and `- _ . '`, and not already exist (ignoring upper and lower case).

Why a crash cannot corrupt a save

`SaveManager.write` first writes everything to `save_N.properties.tmp`, then moves that file over the real one in a single step. If the game dies halfway, the old save is still whole.

When the game saves

- After every successful action in the garden (water, plant, harvest, buy).
- Every 15 seconds while the garden is open.
- When you press SAVE or BACK, and when you close the window.
- Before switching to another garden. If that save fails, the switch is refused.

Errors

Anything that goes wrong becomes a `SaveException` with a sentence a player can read. The screens catch it and show it; a failed save is never ignored silently.

Deleting

DELETE on the Load screen needs two clicks within 3 seconds, so one stray click cannot destroy a garden. If the deleted garden is the one currently open, `GameFrame.deleteSave` closes it first so nothing tries to save to the missing file.

When a flower id in a file no longer exists in the game, loading skips it instead of crashing.

Images

Every picture is a `BufferedImage` made once by `PixelArt` and then reused. A picture comes from one of two places: drawn in code, or loaded from a PNG in `assets/`.

Drawn in code

Methods such as `stem(stage)`, `bud(f)`, `coin()` and `seedBag()` paint rectangles and circles onto a tiny canvas, for example 24x24 pixels. Anti-aliasing is off while drawing. `PixelArt.draw` then enlarges the picture with nearest-neighbour scaling, which keeps the pixels square and gives the chunky look.

Loaded from a file

`PixelArt.loadFile(folder, name)` reads `assets/<folder>/<name>.png` with `ImageIO.read`. If the file is missing or broken it returns `null`, and the caller draws a fallback. A missing picture can never crash the game.

Folder	Used for	If the file is missing
`assets/flowers`	Classmate faces, one PNG per flower id (`klent.png`).	A face is drawn in code.
`assets/icons`	The two potion icons, the two worms, the manure pile.	The drawn icon; no worm or pile.
`assets/menu`	Title logo, ground, farmer and the two cats (loaded by `MenuArt`).	Drawn menu art.
`assets/fonts`	Optional pixel font (`GameRenderer.font`).	Bold Monospaced.

The cache

`PixelArt` keeps two `HashMap`s. One maps a name like `head_daisy` or `stem_2` to its finished picture; the other does the same for loaded files. Each picture is built the first time it is asked for and returned from the map after that. Without this the game would redraw every sprite 30 times a second.

How a flower head is put together

`PixelArt.head(f)` builds one image:

1. Eight petals: circles on a ring, in the flower's petal colour, alternating lighter and darker.
2. A dark disc in the middle.
3. The face on top. If `assets/flowers/<id>.png` exists it is drawn at its own pixel size. If not, `drawnFace` paints skin, hair, eyes, cheeks and a smile from the three values in `FlowerRegistry`.

Two variants are derived from that image pixel by pixel. `wiltedHead` blends every pixel toward brown for a dying flower. `lockedHead` keeps the shape but replaces every colour with dark grey for the collection's silhouettes.

The stem and growth stages are always drawn in code, so a new flower only needs a face.

The grass block

`grassTop(n)` and `dirtSide(n, right)` make small textures, 8 texture pixels per garden tile. Each pixel's colour is picked by `noise(x, y, n)`, a fixed formula that returns the same value for the same position, so the texture looks random but never flickers.

`GardenPanel.paintSlab` stretches those three flat textures onto the top and the two sides of the block with an `AffineTransform`. That is what turns square pixels into slanted ones.

The isometric grid

The garden is a normal grid drawn as diamonds. A plot at row `r`, column `c` is drawn at:

```
x = gcx + (c - r) * tileWidth / 2
y = gtop + (c + r + 1) * tileHeight / 2
```

`updateHover` runs the same formula backwards to find which plot is under the mouse. Plants are painted from the back row to the front row, so nearer plants overlap farther ones.

Small animations

Nothing uses animation frames. Plants sway, ripe flowers bob, worms crawl and clouds drift by feeding the screen's running `time` value into `Math.sin`.

Audio

All sound goes through `AudioManager`, 69 lines using only Java's built-in `javax.sound.sampled`. Any class plays a sound with one line: `AudioManager.getInstance().play("harvest")`.

How a sound is played

1. The name is turned into a file: `assets/sounds/harvest.wav`.
2. The file's bytes are read once and kept in a `HashMap`, so the disk is touched only the first time.
3. A new `Clip` is opened from those bytes and started.

4. A listener closes the clip when it finishes, so clips do not pile up in memory.

Because each play gets its own `Clip`, sounds can overlap: a click and a harvest can be heard together.

Music

`playMusic("bgm")` opens one clip and loops it forever with `Clip.LOOP_CONTINUOUSLY`. `GameFrame` starts it once when the window is created.

Mute

The SOUND button in the garden calls `toggleMute()`. While muted, `play` returns immediately and the music clip is stopped; un-muting resumes the loop.

Why sound can never crash the game

`open()` wraps everything in one `try`. A missing file, an unsupported format, or a computer with no audio device all end the same way: it returns `null` and nothing is played. This is also why the tests can run without speakers.

Which sound plays where

Sound	Played by	When
`bgm`	`GameFrame`	Looping from the moment the window opens.
`click`	`PixelButton`	Every button press.
`water`, `plant`, `fertilize`	`GardenPanel.clickCell`	The action succeeded.
`harvest`, `discover`	`GardenPanel.clickCell`	A harvest; `discover` when it is a new flower.
`buy`	`ItemDialog.buyOne`, `GardenPanel.buyLargerPlot`	A purchase went through.
`error`	Garden and shop	An action was refused, such as planting on dry soil.
`wither`	`GardenPanel.handleEvents`	A flower died; also when a save is deleted.

The ten WAV files are simple generated chiptune placeholders, 16-bit mono. Replacing a sound means replacing the file with one of the same name; no code changes.

Screens and UI

There is one window, `GameFrame`, and it shows one screen at a time. The screens are stacked in a `CardLayout`, and `frame.showGarden()`, `showMenu()` and the others flip to the named card.

Start-up

`Main` asks `SaveManager.getInstance()` for the save folder, creates `GameFrame` (1100x700, minimum 900x600) and shows it. All of this runs inside `SwingUtilities.invokeLater`, because Swing components must be created on Swing's own thread.

One garden screen per save

The menu, How to Play, New Game and Load screens are created once. The garden and collection screens are created again by `GameFrame.startGame` every time a save is opened. A fresh screen holds a fresh `PlayerData`, so nothing from one garden can leak into another.

The frame loop

Every screen extends `ScreenPanel`, which owns a Swing `Timer` firing every 33 ms, about 30 times a second. Each tick does three things:

1. Advances `time`, the number that drives the sways and bobs.
2. Calls `onTick`. The garden uses it to run `PlayerData.update()`, announce flowers that ripened or died, and autosave.
3. Calls `repaint()`.

The timer only runs while its screen is visible. `ScreenPanel.paintComponent` paints the sky and clouds, then calls the screen's own `paintScreen`.

Layout

No layout manager positions the buttons. Each screen overrides `doLayout` and places things as fractions of the window, such as `w * 0.17`. That is why the game still fits when the window is resized.

Custom widgets

Widget	How it works
`PixelButton`	A `JComponent` that paints itself and runs a `Runnable` when clicked. Screens pass method references such as `frame::showMenu`.
`ShopPanel`	Six `PixelButton`s. It receives six `Runnable`s from `GardenPanel` and knows nothing about the rules.
`ItemDialog`	The shop pop-up. It is abstract: it draws the window, the BUY button, the "You got" card and the grid.
`SeedDialog`, `FertilizerDialog`	Fill in the blanks of `ItemDialog`: the title, what BUY does, which items to list, and what clicking one does.

Inside the garden screen

`GardenPanel` keeps which tool is in your hand (`NONE`, `WATER` or `FERTILIZER`) and which plot the mouse is over. It paints in a fixed order: the grass block, the soil tiles, the plants from back to front, the floating "+$25" texts, the bottom bar, the message banner, and the tooltip last so it is on top.

Messages from the rules reach the player through `toast(text, error)`, which shows the banner at the top for 3.6 seconds, red for an error.

Closing the window

The window's close button does not close it directly. It calls `GameFrame.exitGame()`, which saves first. If that save fails, the player is asked whether to exit anyway.

Design patterns

The project claims two Gang of Four patterns: Singleton, used twice, and Factory Method, used once.

Singleton: `SaveManager` and `AudioManager`

A Singleton guarantees one shared object and gives everyone the same way to reach it.

```
private static SaveManager instance;

public static synchronized SaveManager getInstance() {
    if (instance == null) instance = new SaveManager(defaultDir());
    return instance;
}
```

Class	Why there must be only one
`SaveManager`	One object owns the save folder. Two of them writing the same file could overwrite each other's work.
`AudioManager`	One music clip and one mute switch. Two of them would play the music twice, and muting one would not mute the other.

The object is created the first time `getInstance()` is called (lazy), and `synchronized` stops two threads creating two.

One honest detail: `AudioManager`'s constructor is `private`, the textbook form. `SaveManager`'s constructor has no modifier, so only code in the same package can call it. That is deliberate: `SaveFileTest` creates a `SaveManager` pointed at a temporary folder so tests never touch real saves. The game itself only ever uses `getInstance()`.

Factory Method: `ItemFactory`

The shop sells two kinds of random item. The steps are identical: check the money, take the money, create one random item, put it in the inventory. Only "create one random item" differs.

Role in the pattern	In this project
Creator interface	`ItemFactory<T>` with one method, `T create(Random rng)`
Concrete creators	`SeedFactory` returns a `Flower`; `FertilizerFactory` returns a `Fertilizer`
Products	`Flower`, `Fertilizer`
Client	`PlayerData.buySeed()` and `buyFertilizer()`

```
private final ItemFactory<Flower> seedFactory = new SeedFactory();
...
Flower f = seedFactory.create(rng);   // PlayerData does not know how a seed
is rolled
```

Why it helps: `PlayerData` asks for an item without knowing how one is made. To sell a third kind of item, add one new class that implements `ItemFactory`; the buying code keeps its shape.

If the panel asks about other patterns

These are not claimed, but the same ideas appear in the code and are safe to point out:

- **Flyweight-like:** `PixelArt` builds each sprite once and shares it from a cache, instead of one image per plant.
- **Template-Method-like:** `ScreenPanel.paintComponent` fixes the order (sky, clouds, then the screen) and each screen fills in `paintScreen`. `ItemDialog` does the same for its two shops.

Say "similar idea", not "we implemented", unless you are ready to defend it in full.

How to add a flower

Adding a flower is one new line in `FlowerRegistry.java` plus, for a classmate, one PNG. The shop, the collection grid, the save files and the odds all adjust by themselves.

1. Open `src/FlowerRegistry.java` and add a row to the table, in the block of its rarity:

   ```
   flower("mia", "Miamellia", "A camellia that is always early.",
   Rarity.RARE, 0xFF8C42, 0xE8B48A, 0x2B1B12, 1),
   ```

   The eight values are: id, display name, description, rarity, petal colour, then the drawn face (skin colour, hair colour, hair style 0 to 3).
2. For a classmate face, export the face only from Resprite as a transparent PNG, about 35x35 pixels, and save it as `assets/flowers/<id>.png`. The file name must equal the id. Without a file, the game draws a face from the skin and hair values.
3. Update the two checks at the top of `test/GameTest.java` that expect 12 flowers and the 5 / 4 / 3 split, and the count in the README and in the comment above the table.
4. Run the game and the tests.

Things to know before the defense:

- The id is what save files store. Do not rename the id of an existing flower, or saved seeds of it are dropped on load.

- Adding a flower to a tier lowers the chance of each flower in that tier. A fifth Rare makes each Rare 5% instead of 6.25%. The tier's own chance does not change.
- The collection screen shows "x / 13" automatically, because it reads `FlowerRegistry.ALL.size()`.

Other common changes

To change	Edit
Drop chances, sell prices, growth times	The three lines in `Rarity.java`. The chances must total 100 or the game refuses to start.
Seed price, starting money, garden prices	The constants at the top of `PlayerData.java`.
Fertilizer price, bonus or odds	`FertilizerRegistry.java`.
Time to harvest, warning time, autosave interval	`GameConfig.java`, or a `-Dgudotopia...` option without editing code.
A sound	Replace the WAV in `assets/sounds` with one of the same name.
An icon	Put a PNG in `assets/icons` (names are listed in the README there).

Running and testing

The game needs Java 17 or newer and nothing else. It must be started from the project root, because pictures and sounds are read from the `assets/` folder relative to where it runs.

How	Steps
Terminal	`./run.sh` on macOS or Linux, `run.bat` on Windows. Both compile `src/*.java` into `out/` and start `Main`.
IntelliJ IDEA	Open the folder, mark `src` as Sources Root, run `Main.java`. Keep the working directory at the project root.

For the demo, add the VM option `-Dgudotopia.growthScale=0.25` so flowers grow four times faster. To make a flower wither quickly on purpose, add `-Dgudotopia.deathSeconds=8`.

Tests

```
javac -d out src/*.java test/*.java
java -Djava.awt.headless=true -cp out GameTest
java -Djava.awt.headless=true -cp out SaveFileTest
```

Test	Checks	What it proves
`GameTest`	52	The rules: the water-fertilize-plant order, prices, growth and death timing, fertilizer speed, harvest rewards, garden limits, and all three random tables.
`SaveFileTest`	47	Saves: each garden stays separate, everything survives a save and reload, offline growth and death, damaged files, duplicate names, and deleting.
`UiCheck`	not run	Drives the real window with a robot mouse. It needs a free screen, so it has not been run on the current code.

The tests use no test library. Each check prints `PASS` or `FAIL`, and the program exits with an error code if any failed.

Two design choices make these tests possible. `PlayerData` has no Swing code, so it runs without a window. Time and randomness are passed in (`setClock`, the `Random` parameter), so a test can jump 30 seconds ahead or repeat the same dice rolls.

Defense prep

A split for seven members

Each member owns one topic and should be able to open its files and walk through one method. Swap the rows around to suit who knows what.

Member	Topic	Files to know	Guide section
1	Overview, the four layers, the demo	`Main`, `GameFrame`	Start here, File map
2	Game rules and growth timing	`PlayerData`, `GardenPlot`, `GameConfig`	Game mechanics

Member	Topic	Files to know	Guide section
3	The randomizer	`Rarity`, `FlowerRegistry`, `FertilizerRegistry`, `SeedInventory`	The randomizer
4	Saving, loading, deleting	`SaveManager`, `SaveInfo`, `SaveException`, `LoadGamePanel`	Saving and loading
5	Images and the garden drawing	`PixelArt`, `GameRenderer`, `GardenPanel` (the `paint...` methods)	Images
6	Audio and the UI widgets	`AudioManager`, `ScreenPanel`, `PixelButton`, `ItemDialog`	Audio, Screens and UI
7	Design patterns, tests, adding a flower	`ItemFactory`, `SeedFactory`, `FertilizerFactory`, `GameTest`	Design patterns, How to add a flower

Likely questions

Where are the game rules? All in `PlayerData`. The screens call it and show the `Result` it returns.

How does a flower keep growing when the game is closed? The save stores timestamps, not a countdown. On load, `update()` compares them with the current time and applies everything that happened in between.

How is the random seed chosen? Two steps. `Rarity.roll` draws 0 to 99 and walks the 70 / 25 / 5 table. Then one flower of that tier is picked with equal chance.

How do you know the odds are right? `GameTest` rolls 100,000 seeds with a fixed-seed `Random` and checks every tier is within 0.8 points of its percentage.

Why text files and not a database? One small file per garden is enough for this data, needs no library or setup, and can be opened in Notepad to show exactly what is stored.

What if the game crashes while saving? The new data goes to a temporary file first and is moved into place in one step, so the old save stays whole.

What if an image or sound file is missing? The loader returns `null`. A missing picture falls back to art drawn in code; a missing sound is silence. Neither crashes.

Why is `SaveManager` a Singleton? One object must own the save folder, so two writers can never clash. Everything reaches it through `getInstance()`.

Where is Factory Method? `ItemFactory` is the creator, `SeedFactory` and `FertilizerFactory` are the concrete creators, and `PlayerData.buySeed` and `buyFertilizer` are the clients.

Why is the growth time not counted with a timer? A timer stops when the screen changes or the game closes. A timestamp is always right, and it also makes the rules testable with a fake clock.

How would you add a new flower? One row in `FlowerRegistry`, and a PNG in `assets/flowers` if it has a custom face.

Honest limits, if asked

- `GardenPanel` is the largest class. It handles both the mouse and the drawing of the garden; splitting those would be the next refactor.
- Save files are plain text, so a player can edit their money. That is a deliberate trade for being readable.
- Growth trusts the computer's clock. Setting the clock forward grows flowers early.
- The sound files are generated placeholders.
- `UiCheck`, the robot-driven test, has not been run on the final code.